ENGINEERING FIELD MANUAL · A09799

TRACE–QA

Agent Engineering 学习路线与科研项目完整说明书

课程 · 知识图谱 · 口试 · 工程实施 · 评测 · HR 审查

2026-11 · 独立学习工程项目 · 明确区分已完成与待验证功能

01 学习路线与验收

学习路线说明书 · 面向小米 A09799 大模型应用实习生

> 版本 v0.1 · 目标：以可复现的技术能力而非记忆名词来通过技术面试。规划总量约 65–85 小时，6–8 周，每周 8–12 小时；“两周投递版”与“完整作品版”分离。时间为建议，不代表任何能力保证。

0. 先定终点：毕业式验收（Goal/Definition of Done）

完成者必须在不查看答案的条件下，能：

1. **解释和实现**：从空白文件实现一个带工具接口、状态/预算、停止条件、异常处理的 Agent Tool Loop，并能比较规则计划器和模型计划器的责任边界。
2. **搭建检索**：实现 BM25、混合检索、至少一个独立 rerank 模块；对长代码符号、短语、无答案、多跳检索分别提出预测。
3. **评价**：设计经过人工审核的 gold-doc/answerability/answer correctness，计算 Hit@k、MRR、拒答率和任务成功率；对齐版本、模型、prompt、数据集、配置及成本。
4. **诊断**：对失败 trace 说清“检索没命中、证据有但推理错、工具调用失败、预算耗尽、恢复失败”中的哪类瓶颈，执行单变量对照。
5. **工程化**：写自动测试、CLI、README、部署/恢复手册；能明确什么没有验证、哪些数据不可公示。
6. **展示**：自己录制 2–3 分钟演示，现场 20 分钟带问题代码 walkthrough，回答反事实追问而非背定义。

阶段性门槛：两周版本能独立运行 Tool Loop + 检索 + 最小 eval；完整版本至少有公开可复现数据、独立 holdout、真实模型调用、故障注入、对照实验、报告。当前仓库属于**工作型原型**，尚未达到完整研究验收门槛。

1. 基于既有背景的学习诊断（只使用已有可核实经历）

已有经历	可复用长板	无法从简历直接推定的能力
华来科技 LLM 研发中心 Agent 实习	Dify workflow / 多轮对话 / 微调数据准备 / Python 后端	独立 Python Runtime、测试覆盖、可复现 eval、DPO 独立实验
DataWhale 微信智能回复 RAG	RAG 流程和业务问答	Gold labels、混合检索、拒答/引用真实性评价

已有经历	可复用长板	无法从简历直接推定的能力
Kaggle Gemma 4 Developer Agent	Tool Loop、检索、版本控制和评测策略的真实研究素材	本人独立手写所有模块；不能混淆自动化团队成果和个人贡献
无人机跨视角图像检索	特征检索、Recall/mAP 指标、嵌入式部署与性能权衡	文本 RAG 一样的错误分布；两者检索任务不同
SemiValley Python GUI / 系统联调	系统接口与调试耐心	并发服务、CI、权限隔离全栈熟练度

求职叙事目标： “真实企业 Agent 开发经历 + 独立工程的可测量 Agentic Search 项目”。不把全部时间投在 LLM 后训练上：JD 第四项是参与后训练，当前最小补强成本和最高迁移收益在前 1–3 项。

2. 全套节奏与交付物

阶段	时间	必修资源	动手交付	通过条件
P0 Python+Runtime	10h	LangGraph 课程前 3 节；HF Agents Unit 1	无框架 loop + checkpoint/fault tests	能在白板解释状态转移，并手写处理无效参数
P1 Tool/Skill/MCP	8h	MCP 官方架构、Skills 规范	schema 校验+只读 MCP server+回放日志	说清 Host/Client/Server、Skills 运行时定位
P2 Search	14h	Advanced RAG + Hugging Face Agentic RAG	BM25、Hybrid、Rerank + query rewrite	用同一测试集列出新增成功与退步
P3 Evaluation	15h	Evaluating AI Agents、Ragas metrics docs	真实评测协议、gold set、错误归因	不能把检索 Hit@5 冒充回答正确率
P4 Reliability	10h	LangGraph durable execution、故障复盘	crash replay、tool timeout、安全注入	有失败/通过的自动化测试和恢复限制说明
P5 Post-training & research	8–12h	HF TRL SFT / DPO	数据卡、轨迹标注、可选小模型 LoRA	解释何时不需要微调、训练与评测隔离
Final	8–12h	本项目实现说明与答辩题库	Benchmark、独立报告、Demo、简历	可回答 10 个陌生 case + 真实日志证据

两周投递优先版

- **天 1–2：** 核对华来实习本人贡献；运行本仓库，画出 `Runtime→Tool→Retriever→Trace`；做 P0 口试。
- **天 3–4：** 不看实现，手写 `search_docs` schema 与 bounded Tool Loop；注入非法 top_k / timeout。
- **天 5–6：** 对现有 RAG 做 gold-data 修订；手算 Hit@5、MRR；观察 B0/B1 的逐题 gains/regressions。
- **天 7–9：** 实现更真实的 query rewrite 和 reranker；用**公开文档**补测试集。

- **天 10–12：** 针对两个最严重失败归因设计单变量实验；做 2min Demo。
- **天 13–14：** 正式整理投递简历与 20min 技术答辩；不要等待完整研究版才投递。

3. 每节课的学习方法

1. **预注册预测**：在看实现前写 “如果 X 失败，系统应 Y”；示例：“第三次 Tool Call 进程崩溃，重启是否重复？”
2. **只看针对性的讲解**：概念 15–30 分钟，用自己语言画状态图。
3. **动手制造故障**：例如把 `top_k=5` 改成 `top_k=-100`，观察 schema 在哪里拒绝。
4. **记录 Trace**：保存输入、动作、结果、停止原因、耗时、配置哈希。
5. **口试**：先说预期，再解释反例，最后指出怎样证伪自己的结论。
6. **结课 Gate**：通过正确性 + 代码 + 失败归因 + 验证，分数 80/100；安全/正确性缺陷不能靠平均分抵消。

4. 每阶段推荐的认知碰撞案例

- Runtime：Tool Loop 运行 12 轮却从未结束，是 planner 还是 runtime 的责任?
- MCP：Server 挂了，Tool Schema 仍在模型上下文中，是否意味着工具可执行?
- Skill：构建包静态包含脚本，却没在运行时 materialize，为何功能为零?
- Search：`ContextWindowExceededError` 稀有符号用 BM25 反而能更准；什么条件下 Dense 更优?
- Evaluation：B1 命中 19/20，B2 命中 20/20，bootstrap 区间跨 0，如何写结论?
- Reliability：checkpoint 发生在写操作前和后，各有什么重复执行风险?
- Post-training：如果是 tool argument schema 失败，先修数据/接口还是训模型?

5. 每周复盘模板

```
Week / Commit / Runtime configuration:
我原来的预测:
本周最反直觉的一条 trace:
失败类型及证据:
最小可证伪假设:
控制变量与数据集:
观测指标与区间:
新增成功 / 原有成功回归:
下周继续 / 停止的原因:
本人独立实现的代码:
我还解释不清的东西:
```

6. 简历上什么可以说、什么不可以说

- **可以**：使用 Python 构建只读检索 Agent 原型、任务 checkpoint、自动化检索 Benchmark，在固定合成集上评估 BM25/lexical hybrid。

- **不可以**：称当前代码已经实现真实 Embedding 混合检索、全面 MCP 集成、SFT/DPO 训练、实证生产质量或 Kaggle 冲榜成绩。
- **等验证后才能说**：真实模型 Agent 任务成功率、端到端事实正确率、显著性能提升、生产就绪故障恢复。

02 知识图谱

知识图谱 · 从概念到实现到面试

```
flowchart TD
    G[JD: Agent 应用工程师] --> P[Python 工程基础]
    G --> R[Agent Runtime]
    G --> S[Agentic Search]
    G --> E[Evaluation & Iteration]
    G --> T[Post-training: 选修]
    P --> P1[typing/schema/异常/async/test]
    R --> R1[LLM Decision ↔ Tool Execution]
    R --> R2[State / Memory / Checkpoint]
    R --> R3[Budget / Stop / Retry]
    R --> SK[Tool / Skill / MCP]
    S --> S1[BM25: exact symbols]
    S --> S2[Dense embeddings]
    S --> S3[Hybrid / Rerank]
    S --> S4[Query planning / Evidence gate]
    E --> E1[Gold / Task Types]
    E --> E2[Retrieval Hit@k, Recall, MRR]
    E --> E3[Answer Correctness / Grounding]
    E --> E4[Task Success / Trace / Cost]
    E --> E5[Paired A/B / Bootstrap / Regression]
    T --> T1[SFT / LoRA]
    T --> T2[DPO / Alignment]
    R3 --> E4
    S3 --> E2
    S4 --> E3
    E5 --> S4
```

概念边界速记

概念	它解决什么	它不能单独保证什么
Prompt	任务目标与约束表达	可靠执行、权限、可复现
Workflow	预定义流程	开放式任务动态决策
Agent policy / Planner	决定下一动作	工具真正被执行、结果可靠
Runtime	调度执行、状态、预算、恢复	模型推理正确、检索质量
Tool	外部动作契约	安全授权、可用性

概念	它解决什么	它不能单独保证什么
Skill	任务能力封装	运行时安装、数据合法
MCP	Host/Client/Server 的标准协议	自动提供推理、权限、质量
Memory	历史/长期信息	崩溃后 exactly-once 副作用
RAG	取证据用于回答	“有出处”就代表事实正确
Agentic Search	按反馈动态检索	更高成功率 / 更低成本
SFT / DPO	更新模型行为	自动修复工具协议、因果证明
Evaluation	衡量特定数据和指标	自动代表生产环境泛化

四条实际代码路径

- `cli.py → runtime.py → ReadOnlyTools → retrieval.py → Corpus`：一次任务如何运行。
- `CheckpointStore.save/load → events`：状态与证据如何记录。
- `cli.py → evaluation.measure() → paired_compare()`：如何评价改动。
- `llm.OpenAICompatiblePlanner`：模型被限制为“提搜索 query”的明确权限边界。

每学一个节点，需要回答：**它接收到什么、做什么、输出什么、有哪些边界条件、如何测试、失败时谁负责？**

03 课程资料与掌握深度

学习配套资料库 · 课程选择、知识点、深度与阅读作业

> 链接截至 2026-11-02 查验。课程访问形式或付费政策可能变化。只学指定章节，跳过与 A09799 目标低相关的泛化内容。

C1 · Runtime、Tool Loop、状态持久化

主课： DeepLearning.AI · AI Agents in LangGraph（约 1h32m）；必看 Build an Agent from Scratch、LangGraph Components、Persistence and Streaming、Human in the Loop；Essay Writer 作为选修。**辅助课：** Hugging Face Agents Unit 1 的 Tools/Actions/Messages/Agent Workflow。

- **L1 解释：** Workflow 是预定义有向路径；Agent 的“下一步动作”可由模型/策略动态决定；Runtime 负责安全执行、预算和状态，不等于底层 GPU Runtime。
- **L2 代码：** `while steps < max_steps`、状态类型、工具 schema、超时/结果回传、终止条件、idempotent 只读重试。
- **L3 调试：** 在写前 checkpoint 和外部副作用之间发生 crash，理解 exactly-once 不是 SQLite 能自动保证的。
- **本仓库作业：** 阅读 `runtime.py`，画 State→ToolStart→ToolResult→Finish 状态图；改变 budget 和故障注入开关，解释 trace 变化。
- **出口题：** 如果模型把 `top_k` 设成 `true`，Python 的 `bool` 属于 `int` 子类，工具 schema 应如何避免通过?

C2 · Skill 和 MCP

主资源： Agent Skills Specification；MCP Python SDK；MCP 官方 Python first steps。

- **L1 解释：** Tool 是受控动作，Skill 是含说明、脚本和资源的可复用任务包；MCP 的 Host/Client/Server 分工；MCP 有 Tool/Resource/Prompt 三类原语。
- **L2 代码：** 带类型标注的只读 MCP 工具，序列化、异常映射、接口契约测试。
- **L3 调试：** 构建时有 skill≠运行时可访问；服务端工具可被列出≠每次调用都成功；无法恢复的副作用不自动重试。
- **作业：** 为 `skills/incident-triage/` 写一个回归测试，独立启动 MCP Server，把输入 schema 错误与服务端超时分别标记。

C3 · RAG 与 Agentic Search

主课： Building and Evaluating Advanced RAG（约 1h55m）；必看 Advanced RAG Pipeline、RAG Triad、Sentence-window Retrieval、Auto-merging Retrieval。**配套：** Hugging Face Agentic RAG。

- **L1 解释：** 分清 chunking、candidate retrieval、reranking、evidence grounding、answer generation 五个模块；Agentic Search 还允许自主决定是否/如何继续检索。
- **L2 代码：** 手写 BM25 TF/IDF、RRF、TopK、人工 gold-doc 标注；知道 Sparse 与 Dense 的互补性。
- **L3 研究：** 查询改写可能伤害精确代码符号；RRF 可能在高度相关的两个 Lexical Retriever 上带来很小增益。
- **本仓库注意：** `hybrid` 现在是 BM25 + char-gram overlap，**不是 embedding hybrid**。后续一定增加独立 embedding baseline，不能把 RRF 结果写成 Dense 模型效果。
- **出口题：** 为什么 Hit@5 更好但 Answer Accuracy 更差？如何从 Trace 区分召回错/排序错/生成错？

C4 · Eval 与科研方法（最高优先）

主课： Evaluating AI Agents；**技术阅读：** Ragas Context Precision 与其他指标定义。

- **L1 解释：** offline gold set 与 online 用户反馈；可回答 vs 不可回答；检索层、答案层、Agent trajectory 层、运行成本层不能混同。
- **L2 代码：** 每题 `gold_docs`、`answerable`、类别与难度，逐题 paired A/B 输出；Bootstrap 只是描述性不确定性估计，不能自动证明显著性。
- **L3 研究：** 预先锁定 holdout；优化同一小集会过拟合。延迟均值不能替代 P95；抽样时必须保留失败案例。
- **出口题：** 旧版 18/30、新版 21/30，为什么必须看逐题匹配而不只是“多三题”？

C5 · Reliability、Memory、Planning、安全

主文档： LangGraph persistence 和 durable execution。

- **L1 解释：** session memory/store、checkpoint、tool call log、replay 不一样；简单的保存字典≠可靠恢复。
- **L2 代码：** 故障注入、执行预算、timeout/backoff、schema failure fail-fast、允许列表、只读工具与副作用分级。
- **L3 安全：** 检索文档是 untrusted content，注入文本只能作为证据显示，不是高优先级指令；绝不把 API key 交给外部文档。
- **出口题：** Crash 后如何证明没有重复扣费？能否仅凭 SQLite 事务做到？

C6 • Post-training（了解 + 可选小规模实验）

技术文档： Hugging Face TRL SFTTrainer、DPOTrainer。

- **L1 解释：** SFT 的监督轨迹格式 vs DPO chosen/rejected 偏好数据；LoRA 的低秩更新并不是任务成功率保证。
- **L2 实验准备：** 从 Agent trace 采集**已合法授权且脱敏**的成功/失败数据；过滤重复、错误标签、泄漏。
- **L3 评测：** 同架构、同 Prompt/Tool Schema、同冻结 holdout，比较未微调与微调模型；保存训练配置、seed、GPU 时间。
- **出口题：** 错误来自 schema 还是模型策略？为什么很多情况下先调工具和评测比 SFT 更值得？

工具清单与推荐顺序

- 必装：Python 3.11+、Git、编辑器（VS Code）、pytest/unittest、基础 sqlite3（Python 标准库）。
- 推荐：`ruff`、`mypy`、`coverage`、`pre-commit`，用于工程质量控制。
- 进阶：LangGraph、MCP Python SDK、SentenceTransformers（真实 Embedding）、FAISS/Chroma、Ragas、LangSmith/Arize（需检查隐私/成本）。
- 可选训练：Transformers、TRL、PEFT、Accelerate。只在数据标注、伦理与机器资源允许后做。

04 复习口试与实验

口试题库 + 故障卡片 + 评分 Rubric

教练用法： 每个 Part 先随机挑 2 道让学生预测执行行为；让学生直接修改代码复现后，再问“反过来会怎样”。评分四维：机制 25、代码 35、故障诊断 25、证据 15。**不在学生第一次作答前展示参考方向**。标准答案不是背诵模板，而是必须说清验证路线。

Part 0 · Python 工程与接口（L0–L2）

1. `isinstance(True, int)` 为什么为真？如何防止 `top_k=True` 穿过输入验证？
2. `async def` 内调用 `time.sleep` 会怎样？怎么测 event-loop latency？
3. `pytest` 和一次手动运行成功相比，具体防止了什么回归？
4. 重复 `run_id` 的两次请求如何避免覆盖非同一配置的状态？
5. 当 Python 脚本异常退出，`finally` 能保证所有外部副作用被撤销吗？

实验： 将 `top_k=-100`、`top_k=True`、`top_k=100000` 分别发送给工具；写三个失败测试；解释每一种 bug 可能如何破坏线上系统。

Part 1 · Runtime & Memory（L1–L3）

1. 什么是 Agent Runtime？为什么不是 GPU Runtime？
2. Dify 的固定路径 workflow 与模型自主选择下一 Tool Call 何处不同？
3. 工具返回的巨大日志造成 context overrun，应在模型、Runtime、Tool 哪层截断？有什么信息丢失风险？
4. Memory、checkpoint、缓存、conversation history 分别解决什么问题？
5. Agent 在第 3 次工具调用时崩溃，checkpoint 写在调用之前与之后，两种恢复策略分别有何风险？
6. `MAX_STEPS=5` 能保证任务可靠结束吗？如何定义失败终止与善后？
7. 为什么同一 Tool Call 重试安全与否取决于 side effect 和 idempotency？

实验： 在 `tool_start` 事件后故意抛进程异常；重启读取 SQLite，观察 `attempted` 与 `step`；解释为何本原型不是完整 crash-safe replay。

Part 2 · Tool / Skill / MCP（L1–L3）

1. Skill 与 Python function Tool 相比，多了哪些可复用材料？
2. MCP Host/Client/Server 分别处于哪里？MCP Server 是否直接决定模型的下一个 token？

3. Server 暴露出 Tool Schema 等于当前 Tool 可用吗？如何做健康检查？
4. 工具返回非法 JSON，应该让模型自由猜测还是让 Tool Adapter 报 structured error？
5. 为什么系统 prompt 不应信任检索文档里的“忽略一切规则”？
6. Tool 的输入 schema 与权限许可分别属于哪一层？

实验： 把 README 中的 MCP 示例连接到本地读-only 数据；故意关闭 Server；写断连恢复日志（严禁重试会产生外部写操作的工具）。

Part 3 · Retrieval/Agentic Search（L1–L3）

1. BM25 IDF 为什么在精确错误标识符检索中经常有价值？
2. RRF 融合是概率吗？为什么不能把 char-gram 当 Embedding？
3. Query Rewrite 把 `ContextWindowExceededError` 改写成“上下文太长”，可能发生什么？
4. Hit@5 提高但 Answer Accuracy 下降的三个可能原因？
5. 为什么一个最终答案有 citation 不代表这些 citation 支持每一条事实？
6. 哪些检索信号允许 Agent 停止？如果最相关文档也不可靠，如何正确拒答？
7. 为什么 Tool Call 的次数变少不一定说明性能提升？

实验： 人为改变一个 rare symbol，比较 B0、B1 的排名；保存每题排序和错误归因，不要只报告总分。

Part 4 · Evaluation（L1–L3）

1. 旧版 18/30，新版 21/30，如何知道是不是旧对新错互相抵消？
2. bootstrap 95% 区间跨 0 能作什么结论，不能作什么结论？
3. 人工构造 query 和 gold doc，会使实验评价偏向哪些方法？
4. 训练开发集、holdout 集、线上分布有什么本质差别？
5. 何时用 Recall@K，何时用 MRR，何时看正确拒答？
6. 检索得到正确文档但答案仍错误，属于哪层失败？
7. LLM-as-Judge 应如何检查一致性和评分偏差？
8. 模型 A 是 90% 准确率、14s，B 是 88%、5s，如何根据产品约束选择？

实验： 用 `examples/b0_sample.json` 与 `b1_sample.json`，找到唯一的新增命中题，查看 bootstrap 区间；重算不同 K 的结果并解释 sensitivity。

Part 5 · Reliability/Security（L1–L3）

1. 工具超时应该以什么规则重试？与权限拒绝的处理有何不同？
2. 假设报告包含 `IGNORE PREVIOUS INSTRUCTIONS`，怎样证明系统仅把它作为 DATA？
3. 模型输出任意 shell command 的时候为什么需要 allowlist 和人类审批？

4. 恢复 checkpoint 能否保证外部支付恰好一次？需要什么额外设计？
5. 如果缓存里残存测试集的 gold label，研究结果还可信吗？如何防止？
6. 如何通过 trace_id 将一次用户任务的 5 次检索关联起来？

实验： 运行注入文档用例，验证没有任何 shell 执行入口；手动向 checkpoint 写入不合法状态并提出数据库 schema 版本迁移方案。

Part 6 · Post-training/项目答辩（L1–L3）

1. SFT 和 DPO 需要的数据标签分别是什么？
2. 一组 SFT 轨迹包含私有日志时，应该怎样合法处理？
3. 如果 Tool Schema 有 bug，SFT 能修复吗？
4. LoRA loss 降低而 Agent 成功率不变，怎么找原因？
5. 你的项目真正的 innovation 在哪里？如果没有统计显著提升还能发在简历上吗？
6. 你曾让另一个 Agent 自主实现模块，哪些贡献是你亲自设计/编写/验证的？
7. 你这个项目相比于 ordinary RAG Demo 到底多了什么？

三张面试官压力测试卡

Card A — Public score 0.06 → 0.13： V1/V2 有 thinking、output limit、retrieval 改动混杂。问：能否说 thinking 使得成功率翻倍？你要设计什么对照组？

Card B — Skill present but failed: 编译静态把 skill 打进包，但是运行时未实际安装。问：如何构造 smoke canary，证明 Skill 真的执行？

Card C — Context budget overrun: max_output_tokens = max_model_len，第一轮即异常。问：推理 token budget 的固定开销由哪几项组成？如何做 fail-fast？

口试复盘模板

```
题号 / 初始判断 / 信心(0-100)
我写的最小测试或运行记录：
结果与预测冲突：
我调整过的假设：
现在我能解释的最小因果链：
如果换一组参数，我预计：
验证证据文件和 commit：
```

05 课件逐页问题映射

PPT 29 页对应复习问题与动手练习索引

> 使用方式：先看对应页；对每个问题先在纸上写预测；再运行项目或修改参数；记录日志；最后才讨论答案。课程材料中的“预期效果”是实验假设，不是事先保证的结果。

PPT 页	讨论/口试问题（先不看答案）	现场实验或证据
01	系统可运行和可靠完成任务之间最关键的证据差别?	找一次成功与一次无效证据的 Trace
02	你能解释这三个能力如何协作却各自独立吗?	画 3 个模块接口
03	你的华来经历中，哪些是你亲自写代码完成的?	个人贡献清单与源码回顾
04	JD 为什么更重视 P0 而非先做全量 DPO?	取一条 schema failure 做分类
05	为什么先预测比看完教程后再复述更有效?	写下实验前后判断变化
06	Tool 和 Runtime 的错误责任在哪里划线?	改输入参数触发验证
07	LLM 是否负责实际执行 Tool? 为什么?	逐步 trace 一个检索调用
08	ToolStart 后崩溃是否应该重做? 哪些情况下不行?	`test_resume_interrupted_read_only_call`
09	MCP Server 与 LLM 是如何通信的?	画 Host-Client-Server 图
10	文件打包进 Skill 是否证明能用?	设计 canary smoke test
11	BM25、dense、rerank 的角色能否彼此替代?	手算一条 BM25 分数
12	改写精确错误串有何风险?	对比 exact identifier 和语义短句
13	Hit@5、Answer Accuracy、Task Success 为什么不能混用?	读 `evaluation.py` 输出字段

PPT 页	讨论/口试问题（先不看答案）	现场实验或证据
14	多 3 道为什么不等于统计显著的优化?	`traceqa compare` 逐题增退
15	V1→V2 改了多项参数怎么归因?	设计 thinking 控制组
16	权限拒绝和超时为何不应同一种重试?	注入 tool error 并分类
17	Tool schema 出错时应该先训模型吗?	填写问题优先级决策表
18	工程故障问答中的“证据充分”是什么?	制作一个 hard-negative 问题
19	本项目当前 hybrid 为什么不等于 Dense?	看 `_char_overlap()` 源码
20	B0/B1/B2 改了哪些变量？哪些对照不公平?	完成实验矩阵变量表
21	Agent 什么时候继续检索会得不偿失?	预算从 1 改到 3 比调用数
22	checkpoint 保证了原子性为何仍不保证 exactly-once?	回放只读 ToolStart
23	为什么合成集上的 1.00 不能写为业务准确率?	解释 q008，查看 CI
24	如果你只有 1 周，最优先补哪个模块?	指标驱动选择三项动作
25	何时允许冻结一次独立评测集?	列出研究数据泄漏路径
26	如果 HR 说项目就是教程，你怎么用证据回答?	Show code + tests + honest limitations
27	90 秒项目介绍中哪一个指标最可能被追问?	自述并接受随机追问
28	解释 top_k=True、崩溃恢复、引用真实性三题。	白板答辩，立即验证
29	你的“已完成”与“下一阶段计划”如何区分?	完成技术事实清单

随堂评分表

每次只用 100 分：机制 25（准确边界）、代码 35（独立实现）、失败诊断 25（可证伪）、评测证据 15（真实结果）。概念解释可以 100% 正确，但如果代码无法运行，模块仍不算毕业。

教练追问规则

- 不问“背出定义”，而问“输入/状态/输出/反例/测试”。

- 对有信心但答错的问题，先让学生写一段最小测试，再做总结。
- 评分只依据他亲自解释与提供的证据；自动生成代码的存在不等于个人已掌握。

06 项目技术实施

TRACE-QA 工程研究实施说明书

1. 项目定位与成功定义

- **目标 JD**：上海大模型应用实习生 A09799，重点覆盖 Agent Runtime、Skill 开发、Agentic Search、技术方案与可复现实验。
- **用户画像**：研发工程师面对 Issue / traceback / 配置缺陷，需要在文档、故障记录和源码注释中找到**可引用证据**，而不是让模型编造根因。
- **范围内**：只读检索、多轮查询规划、证据出处、失败分类、预算控制、离线评测、可恢复状态、开发文档。
- **范围外（当前）**：自动执行 shell / 修改用户仓库 / 直接生产部署 / 私有数据上传 / 宣称训练了模型。
- **研究贡献目标（尚未完成）**：检验预算和证据门控下的动态检索，是否能以可接受成本改善多步工程任务的可靠性。

2. 总体架构及模块接口

```
CLI/API → Agent Runtime (run_id, max_calls, state, checkpoint)
    → Planner (deterministic replay / opt-in LLM decisions)
    → Tool Registry (strict schema, read-only allowlist)
    → Retrieval (BM25 -> char-gram lexical fusion -> future dense/rerank)
    → Evidence (doc_id + extract, source integrity)
    → Evaluation (gold labels, task success, traces, latency)
    → Reports (paired gains/regressions; champion-challenger decision)
```

重要区别：本仓库 `Agent` 现阶段只在弱词法证据门通过后停止，证据充分性仍然只是很弱的代理条件；B2 不是成熟的 multi-hop Agentic Search。`hybrid` 是两个 lexical 排名，不是 sparse+dense。要实现研究愿景，必须新增独立真正的 embedding 检索和评测人工验证的停止门。

3. 技术栈与为什么这么选

- Python 3.11+, `sqlite3`：兼容招聘期待，轻量离线、清晰理解事务边界。
- 无依赖 BM25 + 字符 ngram：确保 benchmark 在无 GPU/无 API key 情况下可复现。**仅作为 Baseline，不应称 SOTA。**
- RRF rank fusion：用于可解释地融合两个排序器；并不能保证增益。
- JSONL Gold cases + unittest：构建最小科学实验基础。
- Optional OpenAI-compatible planner：LLM 只控制搜索 query；不允许变成 shell / write 权限升级。

- 后续 LangGraph/MCP：目标是对照验证，而不是框架堆砌；MCP 环节要独立真实互操作测试。

4. 仓库中的数据协议

每条 `benchmark.jsonl`：

```
{"id":"q001","query":"QueuePool limit size 5 overflow 10 reached","category":"exact","answerable":true,"gold_docs":["connection-pool.md"]}
```

- `gold_docs` 不来自被测 Agent；由独立审核者依据可追溯信息确认。
- `answerable=false` 时 `gold_docs=[]`；其正确拒答不能用检索返回空列表完全替代（当前仅测 proxy）。
- 每种任务单独报告：精确符号、语义改述、多跳、负例、注入、超时、工具错误。
- 分割：dev 用于探索，holdout 冻结且不读 gold 调参；生产数据必须有公开许可并匿名化。

5. 实验矩阵：必须控制变量

组别	检索	Planner	Evidence Gate	研究问题	当前状态
B0	BM25	无	无	关键词基线	已实现、测试
B1	BM25 + char-gram RRF	无	无	Lexical rank fusion 效应	已实现、测试
B2	B1	bounded rule planner	弱词法覆盖门	Tool Loop 的运行成本及拒答代理	原型实现；不代表成熟 Agentic Search
B3	B1 + Dense / Rerank	LLM query planner	引用充分性+终止预算	多轮动态检索效应	尚未实现
B4	B3	同	+ crash replay & robust safety	可靠性增量	尚未实现
B5	B4	同	同 + SFT/DPO	后训练是否值得	可选研究，尚未实现

反对的研究方式：同时改 top_k、Prompt、模型、chunk size，然后把总体提升归于 Prompt。每轮必须一条主假设，所有配置哈希与逐题输出保留。

6. 可量化评测指标与保护阈值

层级	核心指标	必须避免的错误解释
Retrieval	Hit@5、Recall@5、MRR	不等于 Answer Accuracy
Answer	人审正确率、引用支持率、正确拒答率	当前未测

层级	核心指标	必须避免的错误解释
Agent	task success、loop count、tool error / timeouts	不能仅把少一次 call 当改善
System	P50/P95 端到端时延、token/cost、成功/失败重试	小型合成任务的 CPU ms 无产业意义
Safety	注入成功率、schema 防御、权限违规次数	没有 shell 工具≠所有潜在风险已解决

最低完整研究验收门槛（目标，非当前成绩）：公开合法数据 ≥80 题，独立 heldout ≥25 题，多跳和不可回答各 ≥15 条；对照组设置一致；每项主要指标报告 paired delta、95% 不确定区间、失败例；高风险工具默认关闭；CI 通过；提供运行成本和可复现脚本。绝不预设结果必须上涨。

7. 最有可能产生研究价值的重点方向

重点 A — Evidence-aware Query Budgeting（第一研究重点）

1. 单条查询返回候选后，为每条候选估计与问题的证据匹配（非纯 top-score）。
2. 若 gold 任务要求多步关联，模型必须指出当前缺哪一条依据。
3. 根据“额外查询带来的预期信息增益”决定继续搜索或停止。
4. 将剩余 token、最大调用次数、P95 延迟预算作为停止约束。
5. 与 B1 直接配对比较 `Hit@5`、端到端正确率、成本；验证对高频无答案场景是否适得其反。

重点 B — Failure-aware Tool Recovery（第二重点）

持久化 `tool_intent`、`tool_result`、幂等键；中断后对只读工具重复尝试，对有副作用的操作请求人工确认。做断连、延迟、错误码注入及恢复成功率实验。

重点 C — Calibration & Reliable Abstention（第三重点）

对问题做 answerability 判别，校准置信度，避免低质量依据自信作答。单独设 unanswerable、near-miss、prompt injection 测试集。

不推荐首攻 SFT/DPO：训练很容易吞掉算力、时间、标注资源，却无法自动补齐工具 schema 与评测漏洞。先确认 error taxonomy 中“模型策略能力”占主要比例再决定微调。

8. 项目排期与交付验收

Sprint	范围	具体交付与评审
S0 (已实现)	offline seed	Corpus、BM25/RRF、rule planner、checkpoint、CLI、23 fixture、自动化测试、文档/PPT

Sprint	范围	具体交付与评审
S1	真正模型策略	真实模型 tool-planning trace、schema errors、预算统计、≥10 人工核验任务
S2	Retriever 增强	Dense+Rerank；检索的独立 holdout；每题回归分析
S3	evidence gate	证据门控与多轮 query；真实 human answer annotation
S4	故障恢复	crash at 5 boundaries，MCP 互操作、idempotency tests
S5	冻结与包装	≥80 public tasks；完整报告、CI、GitHub Releases、Demo/video

9. 科研日志要求

每个 Experiment ID 包含 `hypothesis`, `expected_failure`, `baseline_sha`, `challenger_sha`, `model_id`, `prompt_hash`, `corpus_version`, `eval_split`, `random_seed`, `paired_task_ids`, `scores`, `regressions`, `runtime/cost`, `threats_to_validity`, `decision`。

如果结果不升：写失败原理和反例，不修改 gold 标签寻找好看的分数。若证据缺失，填 `UNKNOWN`。

07 首次实验报告

TRACE-QA Baseline Experiment Report · First reproducible fixture

Date: 2026-11-02. **Status:** confirmed local execution, not an independent field benchmark.

Question

Does combining BM25 and character 3-gram lexical rankings using reciprocal rank fusion improve retrieval Hit@5 on our **authored synthetic engineering incident fixture**?

Experiment design

- Data: 12 handcrafted synthetic incident notes; 23 handcrafted queries (20 answerable and 3 unanswerable).
- Baseline B0: BM25 Top-5; Challenger B1: BM25 + character 3-gram overlap RRF Top-5.
- Same corpus, query list, K, code revision, machine environment. **No neural embedding was used**.
- Gold ID annotation was performed while constructing the corpus; therefore strong author/test-set bias exists.
- Evaluation metric: doc-level Hit@5 and MRR; **not** answer correctness, model task success, or independent incident resolution.

Measured results (local execution)

Metric	B0	B1	Interpretation
Hit@5, answerable 20 queries	0.95	1.00	+1/20 query became a hit
MRR, answerable	0.95	0.9667	Several ranking changes, not all strictly positive
Unanswerable correct abstention, 3 cases	0.0	0.0	**Both baselines fail to abstain** by empty-search proxy
Mean retrieval time, ms, local CPU	~0.02	~0.58	Lexical fusion costs more, machine-specific

Paired analysis: **gains=1, regressions=0, unchanged=19**. Approximate paired bootstrap 95% CI for difference in Hit@5: **[0.00, 0.15]**; includes zero, so no robust generalized improvement claim.

The single recovered case is `q008`, a paraphrased "tools keep retrying the same search and use all the budget"; B0's top five missed `agent-loop.md`, whereas B1 ranked it third. This is a concrete example of an observed synthetic benefit, **not proof of a model's reasoning improvement**.

Critical failure revealed

An always-return-top-K retriever returned documents for all 3 deliberately unanswerable queries. The original rule-based planner had a similar weakness: it could treat any retrieved hit as sufficient evidence. As a first engineering response, we added a **weak lexical evidence gate** to the offline runtime. It is intentionally not a calibrated factual entailment test. B2 will be scored separately after the change, and because it alters both search strategy and abstention logic, it cannot isolate which component improves performance.

Next falsifiable experiments

1. **Hard-negative**: add 30 contrastive incidents with lexical overlap but conflicting causes; test evidence gate false positives.
2. **Dense semantic search**: freeze data and add embedding model as a genuinely new retriever; compare both absolute and cost-adjusted ranking.
3. **Independent gold set**: label publicly licensed issues with heldout tasks independent from code authors; measure answer correctness/abstention.
4. **LLM query planner**: compare same retriever with model-driven vs deterministic query selection, with identical budgets and trace metrics.
5. **Crash/replay**: instrument checkpoint at five boundaries; prove idempotent read-only replay and deny non-idempotent tools without approval.

The result is valuable mainly because it establishes the scientific workflow: same-input comparison, failure localization, isolated hypotheses, uncertainty, and honest limitations.

08 运行与调试

技术手册 · 环境、CLI、调试、安全、排错

安装

```
git clone https://github.com/Angusliu6/xiaomi-agent-engineering-intern.git
cd xiaomi-agent-engineering-intern
python -m venv .venv
# Windows PowerShell: .venv\Scripts\Activate.ps1
# macOS/Linux: source .venv/bin/activate
python -m pip install -e .
python -m unittest discover -s tests -v
```

无需 GPU、无需 API key 即可跑离线代码。核心用 Python 标准库；安装/运行命令在 Windows/macOS/Linux 应当兼容。当前没有 Docker 镜像或发布渠道的端到端验证。

复现示例

```
traceqa ask 'MCP search_docs top_k timeout' --budget 3
traceqa ask 'A completely unknown policy XZZZZ' --mode bm25
traceqa eval --variant b0 --out reports/b0.json
traceqa eval --variant b1 --out reports/b1.json
traceqa compare reports/b0.json reports/b1.json
```

输出 `run_id`, `events`, `retrieved`, `citations`, `status`。`evidence_found` 只表示找到相关候选，不代表因果诊断已验证。`insufficient_evidence` 是目前启发式规则，在部分无答案问题可能误判。

Trace 字段读法

- `step`：此次任务中的第几次检索工具执行。
- `tool_start`：调用意图已经记入状态，不保证调用成功。
- `tool_result`：成功返回的候选 doc IDs。
- `tool_error`：类型化失败；当前 sample 支持注入一次 transient error。
- `finish`：状态终止，但证据充分性目前未做人类标注校准。
- `elapsed_ms`：本机本轮时间，不是线上 LLM 延迟，也不等价总任务响应 P95。

常见故障

症状	定位	解决
`ModuleNotFoundError traceqa`	未安装 package	`pip install -e .` 或 `PYTHONPATH=src`
`empty corpus`	目录路径错误	`--corpus data/corpus`；需要 `.md` 文档
`invalid top_k`	负数、过大、bool	检查工具 schema 的输入范围
`resume config mismatch`	不同 query/mode/budget 复用了 run_id	不能跨配置恢复，另起 run_id
`403/timeout`	外部 LLM endpoint 未授权/不可用	先单独测试受信任 API，离线 eval 不依赖它
不可回答被错召回	lexical overlap 引发误匹配	不可把“有结果”当“证据充分”；S3 修 evidence gate

风险与安全政策

- `data/corpus/` 全为演示数据，不应包含工作单位私有材料。
- External LLM 必须显式 `--llm-url` + `--model`，并且只接受 HTTPS 或 localhost HTTP。
- 绝不将 API key 放在命令行或 `git add` 的文件；使用 `TRACEQA_API_KEY` 环境变量。
- 文档里注入的字符串不具有指令权限。当前模型不能调用 shell / write 工具。
- **SQLite 只提供事务式 checkpoint，不实现跨外部服务的 exactly-once；不要拿它执行支付或修改。**
- 如果后续扩展外部 URL 检索，需要防 SSRF、域名 allowlist、body 大小限制与内容净化。

自我学习调试路线

1. 在 `retrieval.Corpus.search` 打断点，亲手计算某个 term 的 `idf`。
2. 在 `runtime.Agent.run` 的 `tool_start` 前后设断点，说明先写事件的理由。
3. 在 `evaluation.measure` 对一题预测 retrieved doc IDs，再执行验证。
4. 修改一个函数，先运行 unit tests，再跑 B0/B1，并找逐题退步。
5. 将观测、预测、修复 commit、回归测试记录到研究日志。

09 简历与面试

技术面试与简历实战手册

90 秒项目陈述（只能在本人真正掌握后使用）

“我本科主要做智能感知和深度学习，也在企业 LLM 研发中心接触过 Dify Agent 工作流与 Python 后端。为了补齐可评测的 Agent 工程能力，我做了 TRACE-QA：一个面向工程故障资料的证据驱动检索系统。它把工具输入校验、任务执行预算、SQLite checkpoint、关键词检索和结构化 trace 分开处理。我首先构建了完全离线可复现的 BM25/RRF 对照以及 23 条合成测试样本。关键是我没有把检索命中率包装成回答正确率；我会区分三类不同的失败：找不到依据、有依据但不会推理、工具执行与恢复出错。目前它还是研究原型，接下来重点验证真正的 LLM 多轮检索、语义检索和独立人工评测。”

2–3 分钟 Demo 演示顺序

1. `traceqa ask 'max_output_tokens equals max_model_len'` → 展示 citations 和 trace；提醒回答是候选证据而非根因判断。
2. `traceqa ask 'completely unknown policy XZZZZ'` → 展示没有证据时的失败行为；指出它仍然存在误召回风险。
3. `traceqa eval --variant b0 --out reports/b0.json` → 固定指标。
4. `traceqa eval --variant b1 --out reports/b1.json` → A/B，展示 `compare` 的 gains/regressions/95% 区间。
5. `python -m unittest discover -s tests -v` → 展示自动化失败测试。

HR 或技术面试官会问的 8 个高压问题

问题	合格回答的结构
“你会 Agent Runtime 吗？”	区分决策与执行，画状态图，解释 bounded loop、schema、checkpoint；现场找到具体源码。
“这个和 Dify 有什么不一样？”	比较可视化流程与 Python 独立控制的 trace/test/错误处理；不要贬低 Dify。
“你做的 hybrid 是 semantic retrieval 吗？”	明确不是：当前 BM25 + lexical char-grams；embedding/reranker 尚是路线。
“为什么得分提高？”	检查同题配对、任务类型、改动变量、区间与反例；拒绝归因过度。

问题	合格回答的结构
“如何防止幻觉？”	检索层/生成层分开、答案证据标注与无法回答策略；目前只输出 extractive evidence。
“你做过 post-training 吗？”	可坦诚说明参与微调数据处理，但未完成独立 DPO；不要说自己训练过未执行的模型。
“Crash recovery 安全吗？”	解释 SQLite checkpoint 的能力边界，副作用工具 exactly-once 不保证。
“项目哪里最差？”	小型合成数据、无真实模型结果、检索层 lexical、无生产安全；优先明确下一次实验。

CV Bullet 规范（需要结果才填数）

- `Designed and implemented a read-only engineering-document retrieval agent in Python, including bounded tool execution, structured traces, and SQLite checkpoints; validated input schema and fault cases with automated unit tests.`
- `Built a reproducible retrieval-only benchmark with annotated document-level relevance labels; compared BM25 and lexical rank fusion with per-case paired analysis and bootstrap uncertainty estimates.`
- **未来完成以后才可补**：`Integrated model-driven search planning and semantic reranking ... achieved [real delta] ... on [held-out task count] ... with [cost/latency].`

个人贡献审查

Kaggle Gemma 4 项目若由 Muse 自动化开发，不应把所有 kernel、harness、LoRA 结果说成“本人独立实现”。面试时用三列：本人设计/理解与审核/其他工具自动生成。任何数字必须附实验配置和原始日志。面试官更重视你能重新复现和解释技术决策。

10 HR 审查与下一步

模拟招聘审核：严苛 HR + Agent Engineer 双审

第一眼会看到什么

优势：企业 LLM Agent 实习经历，与这份 JD 高度匹配；项目有真实 Python 工程源码、文档、自动测试、可运行命令和明确的研究问题。检索/运行时/评测的分层体现出工程认知，不是只做了聊天页面。

立即质疑：这是不是一个只有 12 篇合成文档、23 道题、结果过于容易的教程仓库？所谓 Agentic 是否真的由模型自主规划？“Hybrid” 是不是伪装成 embedding？任意输入是否能误检？能否独立解释 checkpoint 崩溃重放？

当前审查结论：作为教学可运行原型合格，作为“研究级/HR 一眼认定候选人”的**最终旗舰作品尚不合格**。必须完成以下 P0 项，且有新数据支撑后才建议将它放在简历主项目位。

风险级别和必须补齐的项目

优先	现状缺口	对方会怎么追问	必须交付的修补证据
P0	实际多轮/LLM 规划未端到端验证	“为什么称 Agent？”	真实 LLM Tool Call trace + 对照规则策略 + 错误恢复测试
P0	合成数据太简单	“换真实库有效吗？”	公开许可数据 ≥80，holdout ≥25，人工双人抽检
P0	B1 只是 lexical fusion	“有没有做 Embedding？”	Dense 模型 + reranker + 同条件消融
P0	Answer correctness 未测	“没回答对有什么用？”	Gold answer、claim-support 标注、正确拒答独立评分
P0	Evidence gate 过弱	“搜到无关文档也算成功？”	可解释的判据、near-miss 负例、校准曲线
P1	SQLite 不是完整 durable runtime	“crash 半途怎么续？”	五个时点 fault injection 与幂等性/恢复测试
P1	MCP 只在学习说明中	“真正接过 MCP 吗？”	SDK Server/Client 可复现互操作测试
P1	未统计 API token/成本	“线上成本呢？”	P50/P95、tokens/call、预算 tradeoff

优先	现状缺口	对方会怎么追问	必须交付的修补证据
P2	Post-training 无独立成果	“SFT/DPO 经验？”	小模型合法数据训练对照与日志，若时间/算力可用

具体下一次研究如何决策

1. 将 `unanswerable` 错误单独作为最先研究瓶颈（B0/B1 以“检索无结果”为拒答的代理指标仍为 0；B2 的弱词法门对这 3 道合成负例能拒答，但很可能过拟合）。
2. 建立更难的 `hard negative` 集：词高度重叠但根因不同的文档。
3. 让模型选择 search/rewrite/stop，强制每个计划动作都记录 schema 与轨迹。
4. 人工核验 100 条任务，并用冻结集看是否出现回归。
5. 用 paired delta + latency + token cost 共同判断是否提升，不要单凭分数替换 Champion。

招聘流程角度的包装

- Resume headline：**Agent Engineering / Agentic Search / Evaluation**，不称“LLM Training Expert”。
- 2 分钟演示：一次有据可查的成功 + 一次失败 + 一次 A/B（含不确定性）。
- 报告 1 页摘要：问题、主假设、数据、主结果、失败类型、局限、下一实验。
- 如果无法完成完整旗舰项，诚实把 repo 命名为 independent engineering lab，不伪装正式科研成果。